

MedLex — REST API Documentation

Version: 1.0

Base URL: `http://127.0.0.1:8000`

Protocol: HTTP/HTTPS

Format: JSON (all requests and responses)

Table of Contents

1. [Overview](#)
 2. [Authentication](#)
 3. [Token Lifecycle](#)
 4. [Error Handling](#)
 5. [Data Models](#)
 6. [Endpoints](#)
 - [POST /api/accounts/register/](#)
 - [POST /api/token/](#)
 - [POST /api/token/refresh/](#)
 - [POST /api/accounts/auth/google/](#)
 - [GET /api/accounts/profile/](#)
 - [PATCH /api/accounts/profile/update/](#)
 - [PUT /api/accounts/profile/update/](#)
 7. [Frontend Integration \(Angular\)](#)
 8. [Google OAuth Setup](#)
 9. [CORS Configuration](#)
 10. [Configuration Reference](#)
-

Overview

MedLex is a Django REST Framework backend designed for medical professionals. It provides a pure JSON API — no HTML is served. The frontend (Angular or any SPA) communicates exclusively via HTTP requests authenticated with **JWT (JSON Web Tokens)**.

Technology Stack

Component	Technology
Framework	Django 5+ with Django REST Framework
Authentication	JWT via <code>django-rest-framework-simplejwt</code>
Social Login	Google OAuth 2.0 via <code>django-allauth</code> + <code>google-auth</code>
Database	SQLite (development)
CORS	<code>django-cors-headers</code>

Access Token Lifetime

- **Access token:** 60 minutes
- **Refresh token:** 7 days (rotates on every refresh)

Authentication

All protected endpoints require a valid JWT access token in the HTTP `Authorization` header:

```
Authorization: Bearer <access_token>
```

Tokens are obtained via [Login](#) or [Google Sign-In](#).

Authentication Flow

1. Client → POST /api/accounts/register/ (create account)
2. Client → POST /api/token/ (obtain access + refresh tokens)
3. Client → GET /api/accounts/profile/ (authenticated request)
Header: Authorization: Bearer <access>
4. When access token expires →
Client → POST /api/token/refresh/ (get new access token)
Body: { refresh: "<refresh_token>" }

Token Lifecycle

Login / Google Sign-In

→ Returns: { access (60 min), refresh (7 days) }

Use access token
in every request

access token expires?

POST /api/token/
refresh/
→ New access token
+ New refresh token

Note: `ROTATE_REFRESH_TOKENS = True` — every call to `/api/token/refresh/` returns a new refresh token. Discard the old one and store the new one.

Error Handling

All errors follow Django REST Framework's standard response format.

HTTP Status Codes

Code	Meaning
<code>200 OK</code>	Request succeeded
<code>201 Created</code>	Resource created successfully
<code>400 Bad Request</code>	Validation failed; response body contains field-level errors
<code>401 Unauthorized</code>	Missing, expired, or invalid token
<code>403 Forbidden</code>	Authenticated but not permitted
<code>404 Not Found</code>	Resource not found
<code>500 Internal Server Error</code>	Server-side error

Validation Error Format (400)

json

```
{
  "email": ["This field is required."],
  "password": ["Ensure this field has at least 8 characters."],
  "phone_number": ["user with this phone number already exists."]
}
```

Authentication Error Format (401)

json

```
{
  "detail": "Given token not valid for any token type",
  "code": "token_not_valid",
  "messages": [
    {
      "token_class": "AccessToken",
      "token_type": "access",
      "message": "Token is expired"
    }
  ]
}
```

Data Models

User

Field	Type	Constraints	Notes
<code>email</code>	string	Required, unique	Used as login identifier
<code>name</code>	string (max 100)	Required	First name
<code>surname</code>	string (max 100)	Required	Last name
<code>phone_number</code>	string (max 15)	Optional, unique	E.g. <code>+77001234567</code>
<code>organization</code>	string (max 100)	Required	Hospital or clinic name
<code>specialization</code>	string (max 100)	Required	Medical specialty
<code>password</code>	string	Write-only, min 8 chars	Never returned in responses
<code>is_active</code>	boolean	Internal	Defaults to <code>true</code>
<code>is_staff</code>	boolean	Internal	Django admin access

Endpoints

1. Register

Create a new user account.

```
POST /api/accounts/register/
```

Authentication: None required

Request Headers:

Content-Type: application/json

Request Body:

Field	Type	Required	Description
email	string	Yes	Must be a valid, unique email address
name	string	Yes	First name
surname	string	Yes	Last name
password	string	Yes	Minimum 8 characters
organization	string	Yes	Medical organization or clinic
specialization	string	Yes	Medical specialty
phone_number	string	No	Must be unique if provided

Example Request:

```
json
{
  "email": "doctor@hospital.kz",
  "name": "Aibek",
  "surname": "Dzhaksybekov",
  "phone_number": "+77001234567",
  "organization": "City Hospital",
  "specialization": "Cardiology",
  "password": "securePass123"
}
```

Success Response — 201 Created:

```
json
{
  "message": "Account created successfully!"
}
```

Error Response — 400 Bad Request:

```
json
{
  "email": ["user with this email already exists."],
  "password": ["Ensure this field has at least 8 characters."],
  "phone_number": ["user with this phone number already exists."]
}
```

Password Validation Rules:

- Minimum 8 characters
 - Cannot be too similar to personal attributes (e.g. email)
 - Cannot be an entirely numeric password
 - Cannot be a commonly used password
-

2. Login / Obtain JWT

Authenticate with email and password to receive a JWT token pair.

```
POST /api/token/
```

Authentication: None required

Request Headers:

Content-Type: application/json

Request Body:

Field	Type	Required	Description
<code>email</code>	string	Yes	Registered email address
<code>password</code>	string	Yes	Account password

Example Request:

```
json

{
  "email": "doctor@hospital.kz",
  "password": "securePass123"
}
```

Success Response — 200 OK:

```
json

{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Field	Description
<code>access</code>	Short-lived access token (60 minutes). Use in <code>Authorization</code> header.
<code>refresh</code>	Long-lived refresh token (7 days). Use to obtain new access tokens.

Error Response — 401 Unauthorized:

```
json
```

```
{  
  "detail": "No active account found with the given credentials"  
}
```

3. Refresh Access Token

Exchange a valid refresh token for a new access token (and a new refresh token).

```
POST /api/token/refresh/
```

Authentication: None required

Request Headers:

```
Content-Type: application/json
```

Request Body:

Field	Type	Required	Description
<code>refresh</code>	string	Yes	A valid, non-expired refresh token

Example Request:

```
json  
  
{  
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

Success Response — 200 OK:

json

```
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Important: Token rotation is enabled. The old refresh token is invalidated. Always store and use the newly returned `refresh` token.

Error Response — 401 Unauthorized:

json

```
{
  "detail": "Token is invalid or expired",
  "code": "token_not_valid"
}
```

4. Google Sign-In

Authenticate using a Google ID token (issued by Google Sign-In / One Tap). Creates a new user account automatically if the email is not yet registered.

POST /api/accounts/auth/google/

Authentication: None required

Request Headers:

Content-Type: application/json

Request Body:

Field	Type	Required	Description
credential	string	Yes	The raw Google ID token received from the Google Sign-In button

Example Request:

json

```
{
  "credential": "eyJhbGciOiJSUzI1NiIsImtpZCI6Ij..."
}
```

Success Response — 200 OK:

json

```
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "created": true,
  "user": {
    "email": "doctor@gmail.com",
    "name": "Aibek",
    "surname": "Dzhaksybekov",
    "phone_number": null,
    "organization": "",
    "specialization": ""
  }
}
```

Field	Description
<code>access</code>	JWT access token (60 minutes)
<code>refresh</code>	JWT refresh token (7 days)
<code>created</code>	<code>true</code> if a new account was just created; <code>false</code> if the user already existed
<code>user</code>	Full profile object of the authenticated user

Note: When `created` is `true`, `organization` and `specialization` will be empty strings. Prompt the user to complete their profile via [Update Profile](#).

Error Response — 400 Bad Request (no credential supplied):

```
json
{
  "error": "Google credential (id_token) is required."
}
```

Error Response — 401 Unauthorized (invalid/expired Google token):

```
json
{
  "error": "Invalid Google token: Token expired or invalid."
}
```

Error Response — 400 Bad Request (email not in token):

```
json
{
  "error": "Could not retrieve email from Google token."
}
```

5. Get Profile

Retrieve the authenticated user's profile information.

```
GET /api/accounts/profile/
```

Authentication: Required — Bearer token

Request Headers:

```
Authorization: Bearer <access_token>
```

Request Body: None

Success Response — 200 OK:

```
json
{
  "email": "doctor@hospital.kz",
  "name": "Aibek",
  "surname": "Dzhaksybekov",
  "phone_number": "+77001234567",
  "organization": "City Hospital",
  "specialization": "Cardiology"
}
```

Note: `email` is read-only and cannot be changed via the profile update endpoint.

Error Response — 401 Unauthorized:

```
json
```

```
{  
  "detail": "Authentication credentials were not provided."  
}
```

6. Update Profile (Partial)

Update one or more profile fields. Only the fields included in the request body will be modified.

```
PATCH /api/accounts/profile/update/
```

Authentication: Required — Bearer token

Request Headers:

```
Authorization: Bearer <access_token>  
Content-Type: application/json
```

Request Body (any subset of updatable fields):

Field	Type	Required	Description
<code>name</code>	string	No	First name
<code>surname</code>	string	No	Last name
<code>phone_number</code>	string	No	Must be unique; set to <code>null</code> to clear
<code>organization</code>	string	No	Medical organization
<code>specialization</code>	string	No	Medical specialty

`email` is read-only and will be ignored if included.

Example Request (updating specialization only):

```
json
{
  "specialization": "Neurology"
}
```

Success Response — 200 OK:

```
json
{
  "email": "doctor@hospital.kz",
  "name": "Aibek",
  "surname": "Dzhaksybekov",
  "phone_number": "+77001234567",
  "organization": "City Hospital",
  "specialization": "Neurology"
}
```

Error Response — 400 Bad Request:

```
json
{
  "phone_number": ["user with this phone number already exists."]
}
```

Error Response — 401 Unauthorized:

```
json
{
  "detail": "Authentication credentials were not provided."
}
```


7. Update Profile (Full)

Replace all updatable profile fields in a single request. All fields must be provided.

```
PUT /api/accounts/profile/update/
```

Authentication: Required — Bearer token

Request Headers:

```
Authorization: Bearer <access_token>  
Content-Type: application/json
```

Request Body (all updatable fields required):

```
json  
  
{  
  "name": "Aibek",  
  "surname": "Dzhaksybekov",  
  "phone_number": "+77001234567",  
  "organization": "Republican Hospital",  
  "specialization": "Neurology"  
}
```

Success Response — 200 OK:

json

```
{
  "email": "doctor@hospital.kz",
  "name": "Aibek",
  "surname": "Dzhaksybekov",
  "phone_number": "+77001234567",
  "organization": "Republican Hospital",
  "specialization": "Neurology"
}
```

Tip: Prefer `PATCH` for most updates. Use `PUT` only when replacing all profile fields at once.

Endpoint Summary

Method	Endpoint	Auth	Description
POST	/api/accounts/register/	None	Create new user account
POST	/api/token/	None	Login — obtain JWT pair
POST	/api/token/refresh/	None	Refresh access token
POST	/api/accounts/auth/google/	None	Google Sign-In
GET	/api/accounts/profile/	Bearer	Get current user profile
PATCH	/api/accounts/profile/update/	Bearer	Partial profile update
PUT	/api/accounts/profile/update/	Bearer	Full profile replacement

Frontend Integration (Angular)

1. HTTP Interceptor

Attach the JWT token to every outgoing request automatically.

typescript

```
// auth.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getAccessToken();
  if (token) {
    req = req.clone({
      headers: { Authorization: `Bearer ${token}` }
    });
  }
  return next(req);
};
```

Register it in `app.config.ts`:

typescript

```
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from '../auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
};
```

2. Auth Service Example

typescript

```
// auth.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { tap } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private baseUrl = 'http://127.0.0.1:8000/api';

  constructor(private http: HttpClient) {}

  login(email: string, password: string) {
    return this.http.post<{ access: string; refresh: string }>(
      `${this.baseUrl}/token/`, { email, password }
    ).pipe(
      tap(tokens => {
        localStorage.setItem('access', tokens.access);
        localStorage.setItem('refresh', tokens.refresh);
      })
    );
  }

  refreshToken() {
    const refresh = localStorage.getItem('refresh');
    return this.http.post<{ access: string; refresh: string }>(
      `${this.baseUrl}/token/refresh/`, { refresh }
    ).pipe(
      tap(tokens => {
        localStorage.setItem('access', tokens.access);
        localStorage.setItem('refresh', tokens.refresh);
      })
    );
  }

  getAccessToken(): string | null {

```

```
        return localStorage.getItem('access');  
    }  
  
    logout() {  
        localStorage.removeItem('access');  
        localStorage.removeItem('refresh');  
    }  
}
```

3. Google Sign-In Integration

Install the Angular Google library:

```
bash
```

```
npm install @abacritt/angularx-social-login
```

Component:

typescript

```
import { SocialAuthService } from '@abacritt/angularx-social-login';
import { HttpClient } from '@angular/common/http';

export class LoginComponent {
  constructor(private authService: SocialAuthService, private http: Ht
    this.authService.authState.subscribe(user => {
      // user.idToken is the Google credential to send to the backend
      this.http.post('http://127.0.0.1:8000/api/accounts/auth/google/'
        credential: user.idToken
      }).subscribe((response: any) => {
        localStorage.setItem('access', response.access);
        localStorage.setItem('refresh', response.refresh);
        if (response.created) {
          // New user – redirect to profile completion page
        }
      });
    });
  }
}
```

Google OAuth Setup

Step 1 — Create credentials in Google Cloud Console

1. Go to **Google Cloud Console** → **APIs & Services** → **Credentials**
2. Click **Create Credentials** → **OAuth 2.0 Client ID**
3. Application type: **Web application**
4. Add to **Authorised JavaScript origins**:
 - `http://localhost:4200` (Angular dev server)
5. Add to **Authorised redirect URIs**:
 - `http://127.0.0.1:8000` (Django backend, for server-side flows)
6. Copy the **Client ID** and **Client Secret**

Step 2 — Update `settings.py`

python

```
SOCIALACCOUNT_PROVIDERS = {  
    'google': {  
        'APP': {  
            'client_id': 'YOUR_CLIENT_ID.apps.googleusercontent.com',  
            'secret': 'YOUR_CLIENT_SECRET',  
            'key': '',  
        },  
    },  
}
```

CORS Configuration

The API allows cross-origin requests from the following origins by default:

python

```
CORS_ALLOWED_ORIGINS = [  
    'http://localhost:4200',  
    'http://127.0.0.1:4200',  
]
```

To allow additional origins (e.g. staging or production), add them to

`CORS_ALLOWED_ORIGINS` in `settings.py`:

```
python

CORS_ALLOWED_ORIGINS = [
    'http://localhost:4200',
    'http://127.0.0.1:4200',
    'https://medlex.yourdomain.com',
]
```

`CORS_ALLOW_CREDENTIALS = True` is set, which allows the `Authorization` header to be sent cross-origin.

Configuration Reference

JWT Settings (`settings.py`)

Setting	Value	Description
<code>ACCESS_TOKEN_LIFETIME</code>	60 minutes	How long an access token remains valid
<code>REFRESH_TOKEN_LIFETIME</code>	7 days	How long a refresh token remains valid
<code>ROTATE_REFRESH_TOKENS</code>	<code>True</code>	Issues a new refresh token on every refresh call
<code>BLACKLIST_AFTER_ROTATION</code>	<code>False</code>	Old refresh tokens are not blacklisted
<code>AUTH_HEADER_TYPES</code>	<code>('Bearer' ,)</code>	Expected prefix in <code>Authorization</code> header

Quick Start

```
bash
```

```
# Clone and set up the project
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
# API is now available at http://127.0.0.1:8000
```

Requirements

```
Django>=5.0
djangorestframework>=3.15
djangorestframework-simplejwt>=5.3
django-cors-headers>=4.3
django-allauth>=0.63
google-auth>=2.29
```

MedLex API Documentation — generated from source: [medlex_fixed/](#)